

chapter12

제12장 부하 테스트와 최적화

11장의 감시는 “이상이 생겼는가”를 사후에 발견하는 층이었다. 지연 p95가 기준선의 몇 배로 뛰거나 오류율이 오르는 순간을 알려 주지만, 그 순간이 **언제 올지**는 말해 주지 않는다. 골든 시그널의 넷째 — 포화(saturation) — 를 11장이 12장의 몫으로 넘긴 이유가 여기 있다. 트래픽이 지금의 다섯 배·열 배가 되면 이 API는 버티는가, 지연 꼬리(p99)는 어디까지 증가하는가, 먼저 포화하는 것은 CPU인가 메모리인가 I/O인가. 이 질문들은 감시로는 답할 수 없다 — 그런 부하가 실제로 온 적이 없기 때문이다. 12장은 그 부하를 **사전에 만들어** 답한다. 10장에서 배포한 그 모델 API(`models:/complaint_daily_forecaster@champion`)에 Locust로 부하를 걸어, 부하 지표를 정의하고(12.1), 부하를 코드로 기술하고(12.2), 병목 유형을 실측으로 구분하고(12.3), 스케일링의 효과를 워커 증설로 실측한다(12.4). 14장이 “재난 급증 대응은 12장의 사전 실측”이라며 두 번 미뤘던 그 실측이 이 장의 스파이크 시나리오다. 실습(★★★☆☆)은 서빙 앱을 10장 파일의 바이트 동일 복사본으로 두고(재발명이 아니라 재조립), 다섯 가지 부하 시나리오를 한 번에 재현 실행한다.

학습 목표

ML 서비스의 부하 테스트 목적(용량·한계·회귀 발견·회귀 방지)과 핵심 지표를 이해한다.
CPU·메모리·I/O 병목을 증상과 측정 지표로 구분하고 기본 대응을 제시한다.
공공 서비스의 트래픽 급증 상황에서 무엇이 먼저 무너지는지를 실측으로 판단하고 스케일링을 설계한다.

12.1 부하 테스트의 목표와 지표

학습 목표

부하 테스트가 답하는 세 질문(용량·한계·회귀)을 구분한다.
지연을 평균이 아니라 분위수로 보는 이유와 Little's law로 부하를 설계하는 법을 안다.

직관: 감시는 사후, 부하 테스트는 사전

같은 지표(지연·처리량·오류율·포화)를 보더라도 감시와 부하 테스트는 시점이 정반대다. 감시는 실제 트래픽이 임계를 넘는 **그 순간**을 기다려 알려 준다. 부하 테스트는 아직 오지 않은 트래픽을 **지금 인위적으로 만들어** 임계가 어디인지 미리 쟀다. 그래서 부하 테스트의 산출물은 “한계값”이다 — 초당 몇 요청까지 SLO(목표 지연) 안에서 버티는가, 그 이상에서 무엇이 먼저 꺾이는가. 이 값을 모르면 용량 계획도 스케일링 설계도 추측이 된다.

부하 테스트가 답하는 질문은 셋으로 나뉜다.

표 12.1 부하 테스트의 세 목적

목적	물는 것	이 장의 대응
용량 산정(capacity)	목표 부하를 SLO 안에서 견디는가	기준 부하에서 지연·처리량 측정
한계 발견(knee/breaking point)	부하를 올릴수록 어디서 급격히 꺾이는가	단계 증가(ramp)로 무릎점 관찰
회귀 방지(regression gate)	배포마다 성능이 나빠지지 않았는가	같은 부하의 반복 실패를 게이트로

지표: 무엇을 측정하는가

부하 하에서 보는 지표는 11.1의 골든 시그널을 부하의 언어로 다시 쓴 것이다.

표 12.2 부하 지표와 이 실습의 측정 지점

지표	정의	이 실습의 측정
처리량(throughput, RPS)	초당 완료 요청 수	Locust 집계 Requests/s
지연 분위수	p50(중앙값)·p95·p99	Locust 집계 백분위
오류율	실패·타임아웃 비율	Locust failures(+응답 6.0 검증)
포화(saturation)	자원(CPU·메모리)이 얼마나 찼는가	psutil로 서버 프로세스 CPU%·RSS

11.1에서 확인했듯 평균 지연은 일부 요청의 긴 지연을 숨길 수 있다. 4장 Kafka 실측에서 평균 3.5ms 뒤에 최대 110.7ms의 꼬리가 있었듯, 부하 하에서는 “대부분 빠르는데 일부가 느리다”가 규칙이다. 그래서 부하 테스트는 평균이 아니라 분위수를 본다.

오류율도 부하가 오르면 성격이 바뀐다. 평시의 오류가 주로 잘못된 입력(4xx, 10장의 422 거절)이라면, 포화를 넘어선 부하에서는 **타임아웃(timeout)·연결 거부·503(일시적 과부하)**가 나타난다 — 서버가 큐를 감당하지 못해 요청을 늦게 처리하거나 아예 받지 못하는 것이다. 그래서 부하 테스트의 오류율은 “얼마나 실패하는가”만이 아니라 “부하의 어느 지점부터 실패의 종류가 바뀌는가”를 함께 본다. 이 실습에서는 전 구간 오류 0이었으므로(느려지되 죽지 않음), 오류의 성격 변화는 관찰되지 않았고 그 자체가 하나의 결과다 — API는 붕괴가 아니라 완만한 저하(graceful degradation)로 한계에 다가갔다.

워크드 예제: p95는 어떻게 나오는가

분위수 계산은 정렬 한 번으로 확인할 수 있다. 지연 20건이 정렬 후 {2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 7, 7, 8, 9, 11, 14, 30}(ms)이라고 가정한다. p95는 하위 95% 지점의 값 — 위치로는 $0.95 \times 20 = 19$ 번째, 곧 끝에서 두 번째인 **14ms**다(가장 느린 30ms 바로 앞). 평균을 내면 (합 138)/20 = 6.9ms로 “빠른 서비스”처럼 보이지만, 이용자 100명 중 5명은 14ms 이상을, 그중 한 명은 30ms를 기다렸다. 평균이 “전체를 하나로 요약한 값”이라면 p95는 “나쁜 쪽 경험의 경계”이고, p99는 그 경계를 100명 중 1명까지 좁힌 것이다 — 공공 서비스처럼 한 사람의 대기도 민원이 되는 맥락에서는 p99가 SLO의 기준이 되는 일이 많다. (정확한 위치 인덱스와 보간 방식은 도구마다 조금씩 다르므로, 같은 도구로 잰 값끼리 비교하는 것이 원칙이다.)

워크드 예제: Little's law로 부하를 설계한다

부하를 얼마나 걸어야 “동시 요청 L개” 상황이 되는가? 대기행렬 이론의 고전 항등식 하나가 답을 준다 — **Little's law: $L = \lambda \times W$** . L은 시스템 안에 동시에 머무는 평균 요청 수, λ 는 처리량 (정상상태에서 도착률=완료율), W는 평균 지연이다. 포아송·지수 같은 가정이 전혀 필요 없고, 정상상태의 장기 평균이면 성립한다(Little, 1961).

이 실습의 부하 생성은 **닫힌 루프(closed-loop)**다 — 가상 사용자 각자가 대기 없이(요청 간 대기 0) 응답을 받는 즉시 다음 요청을 보낸다. 그러면 사용자 한 명당 항상 정확히 요청 하나가 처리 중이므로, **사용자 수 = 동시 요청 수 L**이 된다. 이 관계 덕분에 Little's law가 곧바로 계산 도구가 된다. 실습의 기준 부하(사용자 20명)에서 처리량 $\lambda = 2498.3$ RPS가 측정되었다면, 평균 지연은 $W = L/\lambda = 20 / 2498.3 \approx 0.0080$ 초 = 8.0ms로 예측된다 — 실측 평균 7.6ms와 사실상 일치한다. 스파이크(사용자 100명)에서는 $W = 100 / 2788.5 \approx 35.9$ ms로 예측되고, 실측 평균은 35.2ms였다. 사용자 수를 다섯 배로 올렸는데 처리량은 거의 그대로이고 지연만 다섯 배가 된 것 — 이것이 포화된 시스템의 특징이며, 12.3에서 그 원인을 자원 지표로 확인한다.

직관: 처리량-지연 곡선과 무릎점

부하를 서서히 올리며 두 곡선을 함께 그리면 이 장 전체가 한 장의 그림으로 요약된다. 부하가 낮을 때는 처리량이 부하를 따라 오르고 지연은 거의 평평하다 — 서버가 여유롭기 때문이다. 어느 지점부터 처리량이 더 오르지 못하고 천장($\lambda_{\max}$)에서 정체된다 — 자원 하나가 포화한 것이다. 그 지점을 지나면 처리량은 그대로인데 지연이 가파르게 오른다(Little's law: L이 느는데 λ 가 고정이니 $W=L/\lambda$ 가 비례해 증가). 두 곡선이 꺾이는 그 부근이 **무릎점(knee)**이고, 시스템의 실효 용량은 무릎점 직전이다 — 그 너머는 “더 많이 처리”가 아니라 “같은 양을 더 느리게”다. 이 장의 실측이 정확히 이 그림이다: 사용자 20→100에 처리량은 약 2500–2800 RPS로 높고 지연만 다섯 배가 되었다. 부하 테스트의 목표는 이 곡선에서 무릎점의 좌표(용량)와 그 원인(포화한 자원)을 찾는 것이다.

현장 방법: 초당 요청 수만으로 용량을 재지 않는다

용량을 “이 서버는 초당 몇 요청(RPS/QPS)까지 견딘다”는 한 수치로 적고 싶은 유혹이 크다. 그러나 대규모 운영의 경험칙은 그 수치에 의존하지 말 것을 권고한다. 구글 SRE의 과부하 처리 지침은 첫 절 제목부터 “The Pitfalls of Queries per Second”이다 — 요청마다 소모하는 자원이 다르기 때문이다. 어떤 요청은 캐시에 적중해 적은 비용으로 처리되고 어떤 요청은 무거운 계산을 유발하며, 같은 엔드포인트라도 입력 크기·클라이언트에 따라 비용이 변동한다. 그래서 구글은 용량을 요청 수가 아니라 **자원 사용률(예: CPU 사용률)**로 재고 그 자원을 기준으로 조달·제한한다(Beyer et al., 2016, Ch. 21). 적용 자격은 배경·일반원칙이다 — 구글은 수천 대 규모의 이야기지만, 이 장은 노트북 한 대에서 같은 원칙을 축소 실측으로 확인한다.

이 장의 역설이 그 확인이다. champion은 상수 예측기라 요청당 계산이 거의 0인데도 처리량은 ≈ 2500 RPS에서 천장을 치고 CPU가 한 코어를 채웠다(12.3). 만약 용량을 “RPS 수치”로만 판단했다면 “초당 2500건까지 가능”이라고 적고 종료했을 것이다 — 그러나 그 천장의 정체(단일 워커의 CPU 포화)는 RPS가 아니라 psutil로 켜 자원 지표(표 12.2의 포화 행)에서만 드러난다. 요청 하나가 무엇을 얼마나 소모하는지를 모른 채 RPS만 계수하면, 요청 성격이 바뀌는 순간(무거운 모델로 교체, 입력 크기 증가) 용량 수치 전체가 무효가 된다. 그래서 부하 보고서의 결론은 “몇 RPS”가 아니라 “몇 RPS에서 어떤 자원이 포화했는가”여야 한다.

민간에서는 이 결론을 계약 문서로 옮긴다. SLA(Service Level Agreement)는 “p99 지연 200ms 이하, 가용성 99.9%”처럼 지연·가용성을 고객·파트너와의 계약으로 명시하고, 실제로 지킨 비율(SLA 준수율)을 정기 보고한다. 계약 수치를 적기 전에 하는 일이 이 장의 부하 테스트다 — SLA를 만족하는 최대 부하를 먼저 재고, 그 부하의 1.5~2배까지 여유를 둔 값을 약속한다. 공공의 SLO도 같은 절차로 정한다. 다른 것은 상대와 대가다 — 계약 상대가 아니라 시민이고, 위반의 대가도 배상금이 아니라 민원과 감사 지적이다. 과부하 때의 선택지도 도메인에 따라 달라진다. 민간은 유료 사용자를 먼저 처리하는 차등을 쓸 수 있지만, 공공에서 같은 차등은 형평성 문제가 된다(12.2).

공공 사례

재난지원금 신청 첫날, 접수 포털이 열리자마자 평소의 수십 배 트래픽이 유입되어 응답이 중단되는 사례는 반복되는 공공 IT 사고다. 사후 보고서는 대개 “동시 접속 폭주”라고 적지만, 그 폭주의 임계가 어디였는지 — 초당 몇 명부터 지연이 SLO를 넘겼는지 — 는 사고 전에 부하 테스트를 했어야만 알 수 있는 값이다. 발주·검수 문서에서 “예상 피크 부하와 그 부하에서의 지연 목표(p95-p99)”를 요구하고, 그것을 부하 테스트 보고서로 확인하는 것이, 개통 첫날의 장애를 개통 전으로 옮기는 유일한 방법이다. - 부하 테스트는 사후 감시가 아니라 사전 실측이다 — 용량·한계·회귀의 세 질문에 “한계값”으로 답한다. - 지연은 평균이 아니라 분위수(p50/p95/p99)로 본다: 평균만으로는 느린 요청의 비율과 최대 구간을 알 수 없으므로 p99를 함께 확인한다. - Little’s law($L=\lambda W$)로 부하를 설계·검산한다 — 닫힌 루프에서는 사용자 수=L이라 $W=L/\lambda$ 가 곧바로 성립한다. - 용량은 RPS 한 숫자가 아니라 “몇 RPS에서 어떤 자원이 포화했는가”로 적는다 — 요청 비용이 저마다 다르므로(SRE Ch. 21).

12.2 Locust 기반 기본 테스트

학습 목표

Locust의 부하 모델(가상 사용자·task·spawn)과 headless·CSV 실행을 안다.
부하 테스트가 성능 이전에 정확성을 검증해야 하는 이유를 안다.

직관: 부하를 코드로 기술한다

Locust는 부하를 “가상 사용자(virtual user)”의 행동으로 기술하는 Python 도구다. 사용자 한 명은 `HttpUser` 이고, 그가 반복하는 동작은 `@task` 로 정의하며, 동작 사이의 대기는 `wait_time` 으로 준다. 실행할 때는 사용자 수(`-u`)와 초당 몇 명씩 투입할지(`spawn-rate`, `-r`), 얼마나 오래(`-t`)를 지정한다. 웹 UI 없이 돌리는 headless 모드에 `--csv` 를 붙이면 통계가 파일로 남아 재현·검증이 가능해진다 — 이 실습이 그 방식을 쓴다. Locust는 Python 3.9 이상에서 실행되고, 이 실습은 3.13에서 검증했다(locust 2.44.4).

`wait_time`으로 부하 강도를 조절한다

같은 사용자 수라도 부하의 세기는 `wait_time` 이 정한다. 요청 사이 대기(이 실습) 사용자가 연속으로 요청해 최대 부하가 되고, `between(1, 3)` 처럼 1~3초 무작위 대기를 주면 실제 이용자의 판단·열람 시간을 모사해 더 현실적인 부하가 된다. 대기를 늘리면 같은 사용자 수로도 도착률(λ)이 낮아지므로, “사용자 1,000명”이라는 수치만으로는 부하 강도가 정해지지 않는다 — 대기 설정이 함께 명시돼야 한다. 이 실습이 대기 0을 택한 것은 한계(포화)를 빠르게 드러내고 Little’s law 검산을 단순하게 하기 위한 선택이며, 평시 부하의 현실성을 재려면 대기를 실제 이용 패턴에 맞춰야 한다.

닫힌 루프와 열린 루프

부하를 거는 방식에는 두 모형이 있고, 이 구분에 따라 결과 해석이 달라진다. **닫힌 루프(closed-loop)**는 고정된 가상 사용자 각자가 응답을 받아야 다음 요청을 보낸다 — 서버가 느려지면 사용자도 덩달아 천천히 보내므로, 시스템에 쌓이는 동시 요청 수가 사용자 수로 묶인다(그래서 사용자 수=동시성 L , 12.1). **열린 루프(open-loop)**는 서버 상태와 무관하게 정해진 도착률(초당 N 건)로 요청을 전송한다 — 서버가 느려져도 요청은 계속 도착해 큐가 무한정 누적되고, 과부하의 붕괴를 더 급격하게 드러낸다. 이 실습은 요청 간 대기를 0으로 둔 닫힌 루프다(사용자 수로 동시성을 통제해 Little’s law 검산이 깔끔하다). 두 모형은 답하는 질문이 다르므로, 부하 보고서에는 어느 모형으로 했는지가 반드시 적혀야 한다.

부하 테스트는 성능 이전에 정확성이다

부하 테스트라고 하면 지연·처리량 같은 성능 수치부터 떠올리기 쉽지만, 그보다 먼저 봐야 할 것이 있다 — **부하 하에서도 답이 옳은가**. 서버가 과부하로 잘못된 값을 반환하거나, 타임아웃 뒤 빈 응답을 200으로 포장해 내보내면, 그것은 느린 것보다 나쁘다. 그래서 이 실습의 locustfile은 응답을

검사한다: 상태 코드가 200이 아니거나 예측값이 6.0(champion의 알려진 출력, 10장 스모크 기준)이 아니면 그 요청을 **실패로 표시**한다. 그 결과 `failures == 0` 은 “오류 없음”과 “전 요청이 옳은 답”을 **동시에** 보증한다. 성능 표를 읽기 전에 이 한 줄을 먼저 확인하는 것이 순서다.

시나리오는 네 모양이다

부하는 한 가지가 아니다 — 어떻게 거는가에 따라 다른 질문에 답한다.

표 12.3 부하 시나리오 네 모양

시나리오	부하 형태	답하는 질문
기준 부하(steady)	고정 사용자 수를 유지	평시 부하에서 지연·처리량은?
단계 증가(ramp)	사용자를 서서히 늘림	지연이 급격히 꺾이는 무릎점은 어디?
스파이크(spike)	순간적으로 급증	급증 시 p99 지연과 회복 시간은?
포화(saturation)	한계까지 부하 증가	무엇이 먼저 포화하는가(자원)?

이 네 모양은 실무의 표준 어휘와 대응한다. 목표 부하에서 SLO를 확인하는 것이 **부하 테스트(load test)**, 한계를 넘겨 붕괴 지점을 찾는 것이 **스트레스 테스트(stress test)**, 급증을 모사하는 것이 **스파이크 테스트(spike test)**, 오래 걸어 누수·성능 저하의 느린 발현을 보는 것이 **내구 테스트(soak/endurance test)**다. 부하 도구들이 정리한 최신 분류도 같은 골격이다 — Grafana k6의 「Load test types」 가이드는 여기에 두 개를 더 세운다: 배포 직후 최소 부하로 “일단 도는가”만 보는 **스모크 테스트(smoke test)**와, 부하를 자동으로 계속 올려 붕괴점의 좌표를 찾는 **브레이크포인트 테스트(breakpoint test)**다(k6, n.d.). 이 실습은 그 스모크의 정신 — “성능을 재기 전에 최소한 옳게 도는가” — 을 전 구간 `failures==0` (부하 중에도 응답이 6.0)으로 이미 갖추고 있고, 짧은 시간의 부하·스파이크에 집중했다(시나리오 0의 sha256 대조는 부하가 아니라 서빙 앱이 10장과 바이트 동일한지 보는 정적 재조립 확인이라 성격이 다르다). 내구 테스트(장시간)와 브레이크포인트(붕괴점까지)는 범위 밖으로 두되 그 필요성은 실습과 운영의 간극에 표기한다. 표준 어휘를 빌려 온 것이므로(방법론 차용) 세부 정의는 도구마다 조금씩 다를 수 있다.

핵심 코드

locustfile의 task — 예측을 호출하고 응답이 여전히 6.0인지 검사한다.

```
with self.client.post("/predict", json=PAYLOAD, catch_response=True) as r:
    if r.status_code != 200:
        r.failure(f"status={r.status_code}")
    elif r.json().get("forecast") != 6.0:  # champion의 알려진 출력(10장 스모크)
        r.failure(f"forecast={r.json().get('forecast')}")
```

전체 코드는 `docs_practice/chapter12/code/12-1-load-test.py` 참고

표기: 부하 생성기도 자원을 소모한다

부하 테스트의 주의점 하나는 **측정 도구가 병목이 되는 것이다**. Locust도 Python 프로세스라 CPU를 소모하고, 부하 생성기와 서버가 같은 machine에서 실행되면 둘이 코어를 경합한다. 생성기가 먼저 포화하면 "서버의 한계"가 아니라 "생성기의 한계"를 재게 된다. 이 실습은 단일 노트북에서 돌리므로 이 교란을 완전히 없애지 못한다 — 그래서 결과의 절대 수치보다 **시나리오 간 상대 비교** (워커 1 대 2, shadow off 대 on)에 무게를 둔다. 운영 규모의 부하 테스트는 생성기를 별도 machine·분산 모드로 실행해 이 교란을 제거한다(보충).

민간의 대비 부하 테스트를 공공의 마감일 스파이크에 적용한다

스파이크 시나리오를 왜 굳이 만드느지는 민간의 관행이 잘 보여 준다. 리테일은 블랙프라이데이, 티켓팅은 예매 오픈, 스트리밍은 대형 경기 중계처럼 **트래픽이 집중될 날짜를 미리 안다** — 그래서 그 날을 겨냥해 예상 배수만큼 부하를 걸어 보는 사전 부하 테스트가 연례 행사가 된다. 목적은 "그날 안 죽는다"를 개통 전에 값으로 확인하는 것이다.

공공에도 같은 날짜가 있다. 다만 성격이 다르다. 연말정산 개통일, 대학 수강신청 오픈 순간, 재난지원금·국민지원금 신청 첫날 — 이들의 공통점은 스파이크가 **수요가 아니라 마감·정원·개시라는 규칙에서 구조적으로 나온다**는 것이다. 민간의 세일은 구매 의사가 있는 사람이 유입되어 어느 정도 시간대로 분산되지만, 법정 마감일과 선착순 정원은 **모두를 같은 순간에 집중시킨다**. 그래서 공공 부하 테스트의 대상 부하는 평균 트래픽이 아니라 그 순간의 피크여야 하고, 예상 배수는 "평소의 몇 배"가 아니라 "마감 1시간에 며칠치가 몰리는가"로 잡아야 한다.

적용이 바꾸는 것이 하나 더 있다. 민간은 과부하 시 일부 사용자를 대기열로 돌리거나 조용히 떨어뜨려도 상업적 손실로 끝나지만, 공공은 형평성·책임성이 걸린다 — 부하를 차단하더라도 "네트워크가 느린 사람, 오래된 기기를 쓰는 사람"이 먼저 배제되는 방식이면 그 자체가 차별이 된다. 그래서 공공의 과부하 대응은 "누구를 버릴까"가 아니라 "모두를 공평하게 느리게(대기 순번의 투명한 안내)"로 설계되고, 성능 실험도 A/B로 이용자를 분할하기보다 새도 배포처럼 이용자 경험을 건드리지 않는 방식이 선호된다(형평성이 민간 기법을 변형시키는 지점 — 12.3·12.4로 이어진다).

공공 사례

지자체가 챗봇 민원 상담을 도입하며 "동시 1,000명 처리"를 요구사항으로 걸었다고 가정한다. 이 수치는 부하 테스트 없이는 검증할 수 없는 주장이다. Locust 시나리오로 사용자 1,000명을 spawn해 지연 분위수와 오류율을 재고, 부하 생성기 자체가 병목이 아님을 확인한 기록 — 그것이 "1,000명 처리"라는 문장의 유일한 증거다. 검수자는 요구 수치가 아니라 그 수치를 만든 부하 테스트 산출물을 봐야 한다. - Locust는 부하를 가상 사용자(HttpUser)의 task로 기술하고, headless+ -- csv 로 재현·검증 가능한 통계를 남긴다. - 부하 테스트는 성능 이전에 정확성이다 — 응답 6.0 검증에 붙여 failures==0 이 옳음까지 보증하게 했다. - 시나리오는 네 모양(기준·단계 증가·스파이크·포화)이며, 부하 생성기 자체의 자원 소모가 측정을 오염시킬 수 있다.

12.3 CPU, 메모리, I/O 병목 구분

학습 목표

세 병목(CPU·메모리·I/O)을 증상과 측정 지표로 구분한다.
“병목은 가정하지 말고 측정한다”를 이 실습의 실측으로 확인한다.

직관: 병목은 가정하지 말고 측정한다

부하를 올리면 어딘가가 먼저 한계에 닿는다. 그 “어딘가”를 추정으로 단정하는 것이 최적화의 가장 흔한 실수다 — 모델 추론이 무거울 것이라 가정하고 GPU를 도입했는데 실제 병목은 로그 쓰기(I/O)였다면, 돈과 시간을 엉뚱한 곳에 쓴 것이다. 10장에서 “실행 환경이 달라야만 드러나는 결함이 있다”며 컨테이너 스모크를 독립 게이트로 둔 것과 같은 규율이 여기서도 적용된다: **병목의 정체는 측정으로만 확정된다.**

표 12.4 병목 유형별 증상·인식·대응

병목	증상	인식 지표	기본 대응
CPU	처리량이 천장에 묶이고 지연만 증가	CPU% 포화(코어 수 ×100%에 근접)	스케일업(코어 ↑)·스케일아웃(워커·노드 ↑)·연산 경량화
메모리	부하 중 RSS 증가·스와핑(swapping)·OOM	RSS가 계속 상승(누수)	배치 축소·모델 경량화·워커당 메모리 산정
I/O	CPU는 여유 있는데 지연·처리량 정체	CPU 낮음 + 쓰기/대기 많음	캐싱·비동기·연결 풀·쓰기 배치

무엇으로 측정하는가는 규모가 정한다. 이 실습은 psutil로 서버 프로세스 하나의 CPU%·RSS를 직접 읽는다. 서비스가 여러 대·여러 계층으로 나뉘면 프로세스 하나만 보서는 병목의 위치를 찾지 못한다 — 민간에서 널리 쓰는 APM(Application Performance Management) 도구는 요청 하나가 게이트웨이(gateway)·API·모델·DB를 거치는 경로를 추적해 어느 구간에서 시간이 갔는지 보여 준다. 측정 대상이 프로세스에서 호출 경로로 넓어진 것이지, “병목은 가정하지 말고 측정한다”는 원칙이 달라진 것은 아니다. 대응의 속도는 다르다. 민간은 병목을 찾으면 클라우드에서 인스턴스를 몇 분 안에 늘리지만, 공공은 장비 조달과 예산 심의에 몇 달을 쓴다 — 사후에 손쓸 여지가 좁은 만큼 사전 측정의 값이 크다.

실측 1: CPU — 단일 워커는 한 코어에 묶인다

이 실습의 champion은 상수 예측기라 예측 연산 자체는 극히 가볍다. 그런데도 부하를 걸면 CPU가 포화한다 — 계산이 무거워서가 아니라, 요청 하나를 처리하는 웹 프레임워크의 작업(요청 파싱·검증·직렬화)이 누적되기 때문이다. 8코어 machine에서 워커 하나(`--workers 1`)로 서빙했을 때, 부하 중 서버 프로세스의 CPU 사용률은 최고 106.3%에 이르렀다 — 100%가 한 코어이므로, **한 워커가 한 코어를 완전히 점유한 것이다.** 결정적 증거는 처리량의 천장이다: 사용자를 20명(기준)→50명(단계

증가)→100명(스파이크)으로 다섯 배 늘리는 동안 처리량은 2498.3→2648.5→2788.5 RPS로 사실상 제자리였고, 늘어난 것은 지연뿐이었다(p50 7ms→15ms→34ms). Little's law가 예고한 그대로다 — 처리량이 천장(λ_{max})에 닿으면, 동시성 L을 올려도 $W=L/\lambda_{max}$ 로 지연만 비례해 늘어난다. 이 시스템의 병목은 모델도 메모리도 아닌 **단일 프로세스의 CPU(파이썬 GIL(Global Interpreter Lock)에 묶인 한 코어)**임이 실측으로 확정된다.

지연은 어디서 늘어나는가: 대기 vs 처리

포화 시 지연이 다섯 배로 늘어난 것을 보고 “모델이 느려졌다”라고 오해하기 쉽지만, 그렇지 않다. 한 요청의 지연은 **큐 대기 시간 + 실제 처리 시간**으로 나뉜다. 이 실측의 처리 시간(상수 예측 + 프레임워크 오버헤드)은 부하와 무관하게 거의 일정하다 — 늘어난 것은 대기 시간이다. 워커 하나가 한 번에 한 요청만 처리하므로, 사용자가 20명이면 평균적으로 앞에 몇 요청이 줄 서 있고, 100명이면 그 줄이 다섯 배 길어진다. p50이 7ms에서 35ms로 오른 것의 대부분은 이 줄서기이지 계산이 느려진 것이 아니다. 따라서 대응은 모델 최적화가 아니라 워커 증설이다. 지연을 대기 시간과 처리 시간으로 나누면 적절한 대응 방식이 달라진다.

CPU 사용률을 어떻게 읽는가

CPU% 수치를 읽으려면 그 척도를 알아야 한다. psutil이 프로세스별로 주는 CPU%는 **한 코어를 100%로 센 값**이라, 8코어 machine에서는 이론상 800%까지 나온다 — 한 프로세스가 여러 스레드로 여러 코어를 쓰면 100%를 넘는다. 그러니 106.3%는 “코어 하나를 완전히 점유하고 아주 조금 넘긴 것”으로 해석해야지 “거의 한계”로 오해하면 안 된다. 이 실측이 서버 프로세스 트리 전체 (uvicorn 마스터+워커)의 CPU%를 합산해 측정하는 이유가 여기 있다: 워커가 둘이면 두 코어를 쓰므로 200% 부근이 정상 상한이고(실측 181.3%), 워커가 하나면 GIL 때문에 100% 부근이 상한이다(실측 106.3%). 척도를 모르면 같은 수치가 “여유”로도 “포화”로도 해석된다.

실측 2: 메모리 — 병목의 부재도 실측 결과다

메모리는 병목이 아니었다. 워커 하나로 돌린 네 시나리오에서 서버 프로세스의 RSS 최고치는 312.3MB 부근에서 안정적이었다 — 부하가 12초·24초 이어져도 계속 상승하지 않았으니, **누수가 없다는 뜻**이다(모델은 시작 시 1회 로드되어 읽기 전용으로 공유되므로, 요청이 누적되어도 메모리가 증가할 이유가 없다 — 10장 lifespan 설계의 효과다). 병목이 없다는 것도 실측으로 말해야 하는 결과다: “메모리는 괜찮다”는 추측이 아니라 RSS 곡선이 평평하다는 관찰이어야 한다.

실측 3: I/O — 요청당 부수 작업의 값을 잴다

I/O 병목의 모양은 **shadow** 로그로 실측했다. 10장 app.py의 shadow 기능을 켜면(challenger 별칭 로드), 요청마다 challenger 예측을 한 번 더 하고 그 결과를 jsonl 파일에 한 줄씩 덧붙인다 — 요청당 파일 쓰기가 붙는 것이다. 같은 부하(사용자 20명)를 shadow off와 on으로 나란히 재자, 처리량이 2498.3 RPS에서 1702.8 RPS로 **32% 감소했다**(비율 0.68). 이때 CPU는 여전히 한 코어 부근(최고 115.4%)에 머물렀다 — 늘어난 부수 작업(추가 예측 한 번 + 파일 쓰기)이 같은 한 코어의 예산 안에서 요청 처리 뭉클을 잠식한 것이다. 정확히 말하면, 이 델타는 순수 I/O만이 아니라 “요청당

부수 작업 전체”의 값이다(추가 예측이라는 약간의 연산이 섞여 있다). 그래도 교훈은 분명하다 — 요청 경로에 side effect(로깅·외부 호출·DB 쓰기)를 하나 붙일 때마다 처리량은 값을 치른다. 그 값을 추정하지 말고 off/on으로 측정한다는 것이 12.3의 요지다.

완만한 저하는 어디까지인가: 부하 차단

12.1에서 이 API가 붕괴가 아니라 완만한 저하(graceful degradation)로 한계에 다가갔다고 적었다 — 부하를 올려도 오류 0, 다만 느려질 뿐이었다. 이 관찰은 정확하지만 오해를 부를 수 있어 한 겹 더 보완해야 한다. **관리되지 않은 과부하는 무한히 완만하지 않다.** 구글 SRE의 연쇄 장애 지침(Beyer et al., 2016, Ch. 22)은 그 다음을 경고한다: 도착률이 처리 천장을 넘어서면 큐가 계속 쌓이고, 큐가 쌓이면 지연이 선형이 아니라 가파르게 폭발하며(Little’s law는 정상상태의 평균 관계인데, 도착률이 처리 용량을 넘으면 그 정상상태 가정 자체가 깨져 큐가 무한히 길어진다 — 12.1에서 $W=L/\lambda$ 가 성립하려면 정상상태여야 한다고 단서를 단 그 지점이다), 지연이 폭발하면 클라이언트 타임아웃→재시도가 부하를 더 늘려 서버가 완전히 정지하는 **연쇄 장애(cascading failure)**로 이어진다. 이 실습이 그 붕괴를 관찰하지 못한 것은 시스템이 견고해서가 아니라 부하 조건 때문이다 — 닫힌 루프(12.2)라 사용자가 응답을 기다렸다 보내므로 큐가 사용자 수 이상으로 폭주하지 못했고, 부하 시간도 12~24초로 짧았다. 열린 루프로 도착률을 천장 위로 고정하거나 부하를 길게 지속하면, 완만한 저하는 어느 지점에서 급격한 붕괴로 바뀐다.

그래서 운영 시스템의 진짜 방어는 “느려지더라도 버티기”를 기대하는 것이 아니라 **부하 차단(load shedding)**을 코드로 넣는 것이다 — 처리 천장을 넘는 요청은 큐에 무한정 쌓지 말고 일찍 503(일시적 과부하)으로 되돌려, 받은 요청만은 SLO 안에서 끝내고 나머지는 빠르게 거절한다. 역설적이지만, 일부를 의도적으로 거절하는 것이 전부를 느리게 만들어 연쇄 장애로 가는 것보다 낫다. 이 실습의 app.py에는 그 차단 로직이 없다(범위 밖) — 그래서 12.1의 “완만한 저하”는 이 부하 구간에서의 관찰이지 무한한 보증이 아니라는 단서를 여기 남긴다.

공공 사례

민원 예측 API에 “모든 요청을 감사 로그로 남기라”는 규정이 붙는 것은 공공 시스템에서 흔하다 — 정당한 요구다. 그러나 그 로깅을 요청 경로에서 **동기적으로** 파일·DB에 쓰면, 이 실습에서 확인한 대로 처리량이 3분의 1씩 감소할 수 있다. 대응은 로깅을 없애는 것이 아니라 요청 경로에서 빼는 것이다 — 비동기 큐로 넘기거나 배치로 모아 쓰면 감사 요건과 처리량을 함께 지킨다. 규정과 성능이 부딪칠 때, 부하 실측은 “얼마나 부딪치는지”를 수치로 만들어 설계 논의를 감정이 아니라 값 위에서 하게 한다. - CPU 병목은 실측으로 확정됐다: 단일 워커가 한 코어를 채우고(106.3%), 처리량이 ≈ 2700 RPS 천장에 묶여 지연만 늘었다. - 메모리는 병목이 아니었다 — RSS가 312.3MB 부근에서 평평(누수 없음). 병목의 부재도 실측 결과다. - I/O(요청당 부수 작업)의 값은 off/on으로 켜다: shadow on에서 처리량이 0.68배로 떨어졌다. - 완만한 저하는 이 부하 구간의 관찰일 뿐 — 관리되지 않은 과부하는 어느 지점부터 붕괴하므로, 운영은 부하 차단(503 조기 거절)으로 방어한다 (SRE Ch. 22).

12.4 스케일링 전략 개요

학습 목표

스케일업과 스케일아웃을 구분하고, 상태 없는 서비스가 수평 확장에 유리한 이유를 안다.
워커 증설의 효과와 한계를 실측으로 이해한다.
스케일링 결정의 어느 부분이 도메인과 무관하고 어느 부분이 근거만 바뀌는지 구분한다.

직관: 위로 키우거나, 옆으로 늘리거나

병목이 CPU로 확정되면(12.3), 대응은 두 방향이다. **스케일업(수직)**은 더 큰 machine — 코어·메모리를 키운다. 단순하지만 상한이 있고 비싸다. **스케일아웃(수평)**은 같은 서비스를 여러 개 실행한다 — 프로세스·컨테이너·노드를 늘린다. 수평 확장이 싸려면 서비스가 **상태 없음(stateless)**이어야 한다: 요청을 아무 인스턴스가 처리해도 결과가 같아야 자유롭게 복제·분산할 수 있다. 이 실습의 서빙 API는 모델을 읽기 전용으로만 사용하므로 stateless다 — 그래서 워커를 늘리면 된다.

왜 프로세스를 늘려야 하는가: GIL

파이썬에는 GIL(Global Interpreter Lock)이 있어, 한 프로세스 안에서는 여러 스레드가 있어도 파이썬 바이트코드를 **한 번에 하나씩만** 실행한다. 그래서 CPU를 쓰는 일(12.3에서 본 요청 파싱·검증·직렬화)은 스레드를 아무리 늘려도 한 코어를 넘지 못한다 — 12.3에서 워커 하나의 CPU가 106.3% (한 코어)에서 멈춘 근본 원인이다. GIL의 벽을 넘는 길은 스레드가 아니라 **프로세스**다: 프로세스마다 독립된 인터프리터와 독립된 GIL을 가지므로, 워커를 N개 띄우면 N개의 코어를 동시에 쓸 수 있다. 12.4의 스케일아웃이 스레드 증설이 아니라 프로세스(워커) 증설인 이유가 여기 있다.

실측: 워커 하나 대 둘

uvicorn의 `--workers N` 은 서빙을 N개의 프로세스로 실행한다. 프로세스마다 독립된 파이썬 인터프리터(독립 GIL)가 있으므로, 12.3에서 본 “한 코어에 묶임”을 프로세스 수만큼 풀어낸다. 같은 기준 부하(사용자 20명)를 워커 1개와 2개로 재자 —

표 12.5 부하 테스트 결과 해석표(실측 스냅샷 — 지연·RPS·CPU는 재실행마다 변동)

시나리오	워커	사용자	RPS	p50	p95	p99	평균	CPU 최고	RSS 최고
기준 (steady)	1	20	2498.3	7ms	11ms	18ms	7.6ms	106.3%	312.3MB
단계 증가 (ramp)	1	50	2648.5	15ms	21ms	32ms	14.5ms	107.2%	313.5MB
스파이크 (spike)	1	100	2788.5	34ms	42ms	66ms	35.2ms	106.9%	314.1MB
스케일 (scale)	2	20	3153.0	6ms	9ms	12ms	5.6ms	181.3%	610.1MB
I/O(shadow on)	1	20	1702.8	11ms	14ms	26ms	11.4ms	115.4%	311.2MB

워커를 둘로 늘리자 처리량이 2498.3→3153.0 RPS로 올랐다(1.26배). 지연도 이 스냅샷에서는 소폭 개선됐다 — p50은 7ms→6ms, 꼬리 p99는 18ms→12ms로 내렸다. 다만 사용자 20명이라는 낮은 동시성에서는 이미 지연이 낮아, 워커 증설의 지연 개선 폭은 작고 실행마다 변동한다(재실행하면 지연은 개선되기도 거의 불변이기도 하다 — 그래서 워커 증설에서 **확실히** 얻는 이득은 지연 감소가 아니라 처리량 천장의 상향이다). CPU 최고치는 106.3%에서 181.3%로 — 한 코어에서 두 코어 쪽으로 늘었다. RSS는 312.3MB에서 610.1MB로 약 두 배가 되었는데, 이는 누수가 아니라 **워커마다 모델을 따로 로드**하기 때문이다(스케일아웃의 메모리 비용 — 인스턴스 수만큼 곱해진다).

표기: 왜 정확히 두 배가 아닌가

처리량이 1.26배에 그친 것(2배가 아니라)은 이 실측의 조건 때문이다. 첫째, 동시 사용자가 20명뿐이라 두 워커에 나눌 offered load 자체가 작다 — 워커가 놀 여유가 생긴다. 둘째, 부하 생성기(Locust)도 같은 8코어 machine에서 코어를 경합한다(12.2의 교란). 워커 증설의 효과를 온전히 보려면 사용자 수를 함께 올리고 생성기를 분리해야 한다(스케일 시나리오도 사용자 100명으로 걸었다면 처리량 상승 폭이 더 컸을 것이다). 그럼에도 방향은 분명하다 — 워커를 늘리면 처리량 천장이 오른다. 스케일링의 **효과의 부호**는 실측으로 확인됐고, **크기**는 부하와 환경에 달렸다.

이 1.26배를 비용의 언어로 옮기면 판단 기준이 하나 더 생긴다. 워커·인스턴스를 두 배로 늘리면 비용도 대체로 두 배가 되는데 처리량은 1.26배만 올랐다 — 민간은 이 격차를 비용 대비 성능(cost-performance), 곧 비용 1원당 처리 요청 수의 문제로 해석한다. SLO를 지키면서 그 값을 가장 높이는 운영점이 12.1의 무릎점 직전이다. 그 너머로 부하를 증가시키면 처리량은 더 오르지 않고 지연만 저하되며, 훨씬 아래에서 운영하면 유휴 자원에 예산을 소모한다. 공공에서도 계산은 같고 결과가 가는 곳만 다르다 — 비용 절감 보고서가 아니라 예산 심의 자료다.

워크드 예제: 워커를 몇 개까지 늘릴 수 있는가

워커 증설의 상한은 두 자원이 정한다 — 코어와 메모리. 이 실습의 8코어 machine에서 워커는 코어 관점으로 최대 8개까지가 의미 있다(그 이상은 코어를 분할 사용해 프로세스 전환 비용만 증가한다). 메모리 관점의 상한은 실측에서 나온다: 워커 2개의 RSS가 610.1MB였으니 워커 하나는 약 $305\text{MB}(=610.1/2)$, 8개면 약 $2.4\text{GB}(=8 \times 305)$ 가 필요하다. 만약 이 서버의 메모리가 2GB뿐이라면 코어가 남아도 워커는 6개 부근에서 메모리에 먼저 막힌다 — 두 상한 중 **먼저 닿는 쪽**이 실제 상한이다. 그래서 용량 계획은 “코어 수”만이 아니라 “워커당 메모리 $\times$ 워커 수 $\leq$ 가용 메모리”를 함께 풀어야 하고, 두 값 모두 추정이 아니라 이 장치럼 실측에서 얻는다.

한계와 그다음

한 machine의 두 상한(코어·메모리)에 닿으면 다음은 여러 machine으로의 수평 확장이고, 트래픽에 따라 인스턴스 수를 자동으로 조절하는 것이 오토스케일(autoscaling)이다(Kubernetes HPA — CPU·메모리 기반 또는 지연·큐 길이 같은 custom metric 기반, 보충). 컨테이너 환경에서는 한 컨테이너에 워커를 여럿 두기보다 컨테이너(파드)를 여러 개 복제하는 편이 권장된다 — 10장에서 본 “이미지= 완결된 배포 단위”가 확장 단위도 되는 것이다. 인스턴스가 여럿이 되면 요청을 어느 인스턴스로 보낼지 나누는 **로드밸런서**가 앞단에 필요한데, 서비스가 stateless라면(이 API처럼) 어느 인스턴스가 받아도 답이 같으므로 분배 규칙이 단순해진다(라운드로빈(round robin)으로 충분) — 상태를 요청 경로에 두지 않은 설계가 확장 단계에서 이득을 주는 지점이다.

오토스케일을 쓰는 이유의 절반은 비용이다. 피크 부하를 항상 감당할 만큼 인스턴스를 상시 유지하면 평시에는 대부분이 논다 — 평시에 작게, 급증 시에만 크게 운영하면 그 낭비가 줄어든다. 대신 확장에는 시간이 걸린다. 새 인스턴스가 기동하고 모델을 로드하는 동안에도 요청은 계속 들어오므로, 그 구간의 지연 급등을 스파이크 시나리오로 미리 재 두어야 한다. 급증 시각을 미리 아는 경우 — 공공의 마감일·개시일, 민간의 세일 개시 시각 — 예는 CPU 임계를 뒤따라가는 자동 확장보다 그 시각에 맞춘 예약 확장(scheduled scaling)이나 사전 워밍(pre-warming)이 그 구간 자체를 없앤다.

14장 회수: 재난 급증에서 먼저 포화하는 지점

14장은 통합 MVP를 만들며 “재난 시 트래픽이 평시의 수십 배가 되면 어디가 먼저 무너지는가 — 소비 지연인가 API 포화인가 — 는 추측이 아니라 부하 실측의 문제”라며 그 실측을 12장으로 미뤄 두었다. 이 장의 스파이크 시나리오가 그 답의 절반이다: API 쪽은 사용자 100명 급증에서도 오류 0건으로 버티되(정확성 유지), 처리량 천장(≈ 2800 RPS)에 막혀 지연 꼬리가 p99 66ms(최댓값 123.3ms)까지 늘어났다 — API의 한계는 “중단”이 아니라 “지연 증가”로 나타났다. 나머지 절반(Kafka 소비 지연)은 14장 스택의 파티션·컨슈머 문제이며, 두 실측을 합쳐야 재난 급증의 전체 그림이 된다. 스케일링 설계는 그래서 API 워커 증설만이 아니라 파티션 확장까지 함께 가야 한다.

용량 조달은 측정 다음이다(17.3 회수)

워커 증설은 단순한 성능 튜닝이 아니라 **용량 조달(capacity provisioning)**의 최소 단위다. 얼마나 조달할지는 두 방향의 실패 사이에서 정해진다 — 과소 조달은 마감일 스파이크에서 장애로 터지고, 과잉 조달은 놓고 있는 워커·인스턴스에 예산을 태운다. 이 둘 사이의 옳은 지점은 짐작이 아니라 이 장 같은 실측에서만 나온다. “측정하고 나서 증설한다”는 순서가 곧 FinOps(재무·운영 결합)의 기본기이고, 17.3이 그 순서를 이 장의 수치로 되받는다.

구체적으로, 17.3의 비용-용량 계산은 이 장이 만든 실측을 입력으로 쓴다: 단일 워커의 처리량 천장(기준 부하 2498.3 RPS), 스파이크에서의 천장(2788.5 RPS), 단일 워커가 한 코어를 완전히 점유한 CPU 포화(기준 부하 106.3%, 표 12.5), 워커를 하나 더 붙였을 때의 처리량 증가분(1.26배), 요청 경로에 side effect를 붙였을 때의 처리량 손실(0.68배). 이 값들이 있어야 “예상 피크를 SLO 안에 넣으려면 워커가 몇 개, 인스턴스가 몇 대 필요하고 그 비용이 얼마인가”를 계산할 수 있다. 값이 없으면 조달량은 벤더의 권장 사양이나 담당자의 경험적 판단으로 정해지고, 그 판단은 개통 첫날 트래픽으로 처음 검증된다. 부하 실측이 성능 장의 산출물이면서 동시에 예산 장(17.3)의 입력인 이유가 여기 있다 — 성능과 비용은 같은 측정 결과를 서로 다른 관점에서 해석한 것이다.

공공 사례

“재난 시스템”을 표방하는 공공 서비스라면, 평시가 아니라 급증 시의 한계가 검수 대상이어야 한다. 이 장의 방법이 그 검수의 언어를 제공한다 — 예상 급증 배수만큼 부하를 걸어 오류율·지연 꼬리·자원 포화를 재고, 스케일링(워커·인스턴스·파티션)으로 한계를 어디까지 밀 수 있는지 실측한 보고서. 그 보고서 없이 “재난 대응”을 적은 제안서는 검증되지 않은 주장이다(14장 범위표가 이 점을 명시했다).

역방향: 이 장의 부하 실측을 민간 현장에 옮기면

지금까지는 공공의 마감일 스파이크를 대상으로 부하를 설계했다. 반대 방향도 성립한다 — 이 장의 절차를 그대로 민간 현장에 적용하면 무엇이 바뀌고 무엇이 유지되는지 세 현장에서 확인한다.

SLA 기반 용량 계획과 클라우드 비용 산정은 12.1의 Little’s law를 거꾸로 쓰는 현장이다. “p99 지연 100ms 이하를 동시 1만 명에서 유지”라는 계약 수치가 먼저 주어지고, 목표 W (지연)와 예상 λ (처리량)에서 필요한 L (동시성=서버 용량)을 역산한다. 거기에 워커당 메모리 실측(앞의 워커 상한 계산)을 없으면 필요한 인스턴스 수가 나오고, 인스턴스 수에 클라우드 단가를 곱하면 월 인프라 비용이 나온다. 공공에서 같은 계산의 결과가 예산 요구서로 갔다면(17.3), 여기서는 비용 산정서로 간다 — 계산식은 같고 수신처가 다르다.

모델 경량화의 판단은 12.3의 병목 구분이 없으면 시작하지 못한다. 추론을 ONNX 같은 형식으로 변환해 CPU당 처리량을 올리는 작업은 병목이 CPU 연산일 때만 효과가 있다 — 병목이 I/O였다면 모델을 아무리 줄여도 처리량은 그대로다. 그래서 순서가 “경량화 후 측정”이 아니라 “측정 후 경량화”이고, 변환 전후로 같은 부하를 걸어 처리량 비율을 측정하는 것이 이 장의 shadow off/on 대조(12.3)와 똑같은 형태다. 바뀌는 것은 그 비율을 읽는 언어다 — 공공은 “같은 예산으로 더 많은 민원을 처리”, 민간은 “같은 비용으로 더 많은 요청을 처리”.

자동 확장 정책 검증은 스파이크 시나리오를 확장 규칙이 켜진 상태에서 다시 돌리는 것이다. “CPU 70% 초과 시 인스턴스 추가”라는 규칙이 실제로 몇 초 만에 발동하는지, 그 사이 p99가 어디까지 치솟는지는 규칙 문서가 아니라 실측이 답한다. 이 장이 워커 1 대 2를 대조한 자리에 확장 off 대 on이 들어가는 것 — 대조의 구조는 같다.

공통 원칙. 무엇을 측정하는가(지연 분위수·처리량·오류율·포화), 어떻게 거는가(단히 루프 대 열린 루프, 네 시나리오), 무엇이 포화했는가(CPU·메모리·I/O), 얼마나 늘릴 것인가(코어·메모리 두 상한) — 이 네 결정은 어느 현장에서든 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 형평성·법정 마감일·예산 심의라면, 민간에서는 매출·SLA 위약금·클라우드 비용이다. 방향은 비대칭이다. 공공용으로 쓴 부하 보고서 — “몇 RPS에서 어떤 자원이 포화했는가”와 그 근거 — 는 민간에 그대로 내도 충분하지만, 민간의 “초당 몇 건 처리 가능”이라는 한 줄 결론을 공공 검수에 내면 근거가 비어 있다.

스케일업(수직)과 스케일아웃(수평)은 다르다 — stateless 서비스는 워커·인스턴스를 늘리는 수평 확장이 싸다.

워커 증설은 용량 조달의 최소 단위다 — “측정 후 증설”의 순서로 과소 조달(마감 스파이크 장애)과 과잉 조달(예산 낭비) 사이를 실측으로 정한다(17.3 FinOps 회수).

실측: 워커 1→2에서 처리량 2498.3→3153.0 RPS(1.26배), CPU는 한 코어→두 코어, RSS는 두 배(워커마다 모델 로드) — 지연은 이 스냅샷에서는 소폭 개선(p50 7→6ms)됐으나 낮은 동시성이라 변화 폭이 작고 실행마다 변동한다(확실한 이득은 처리량 천장 상향).

재난 급증에서 API는 중단되지 않고 지연만 증가했다(스�파이크 p99 66ms, 오류 0) — 스케일링은 워커 증설과 파티션 확장을 함께 설계한다(14장 연결).

무엇을 재고·어떻게 걸고·무엇이 포화했고·얼마나 늘릴지라는 네 결정은 도메인과 무관하게 똑같이 내리고, 근거만 바뀐다 — 공공은 형평성·마감일·예산 심의, 민간은 매출·SLA 위약금·클라우드 비용.

실습 12.1 모델 API 부하 테스트 실행(★★★☆☆)

실습 목표

10장에서 배포한 그 서빙 API에 다섯 가지 부하 시나리오를 걸어 지연·처리량·포화를 실측한다.

병목을 실측으로 구분하고(CPU·메모리·I/O), 워커 증설의 효과를 재며, 결과 해석표와 개선 우선순위표를 산출한다.

입력 데이터·서빙 앱에 대한 표기

부하를 거는 대상은 새로 만든 API가 아니라 **10장의 그 API**다 — 서빙 앱(app.py)은 10장 파일의 바이트 동일 복사본이고, 오케스트레이터가 sha256 일치(ec7ec653...)를 실행 첫머리에 확인한다 (14장의 재조립 확인 기법 재사용). 모델은 bootstrap이 9·10장과 같은 데이터·같은 절차로 chapter12 로컬 레지스트리를 재구성해 조달하며, 같은 데이터임은 훈련 데이터 지문 96f6b6b3... (9장 기록과

일치)을 코드가 강제 확인한다 — 부하의 대상이 앞 장과 다른 모델이었다면 실측의 의미가 없기 때문이다. 환경변수 이름에 `CH10_` 접두가 남아 있는 것은 `app.py`를 한 글자도 바꾸지 않은 바이트 동일 복사의 결과다.

선수 조건과 실행

Python 3.13에서 검증했다(locust 2.44.4-psutil 7.2.2-uvicorn 0.50.2-mlflow 3.14.0-scikit-learn 1.9.0). 별도 서버·컨테이너·API 키가 필요 없다 — bootstrap이 로컬 레지스트리를 재구성하고, 오케스트레이터가 서버를 직접 기동하고 종료한다.

```
cd docs_practice/chapter12
python3 -m venv venv && source venv/bin/activate # (Windows: venv\Scripts\activate)
pip install -r code/requirements.txt
python run_chapter12.py
```

시나리오	내용	대응 절
0	sha256 대조 — 서빙 앱이 10장 파일과 바이트 동일한가	실습 서두
1	bootstrap — 레지스트리 재구성 (지문 확인), champion=v1(6.0)	12.1
2	baseline(워커 1, shadow off) — 기준 부하 + CPU/메모리 샘플링	12.1, 12.3
3	ramp(워커 1) — 단계 증가로 무류점 관찰	12.1, 12.2
4	spike(워커 1) — 재난 급증 시 p99 지연·회복	12.2, 12.4
5	scale(워커 2) — 워커 증설의 처리량 델타	12.4
6	io(워커 1, shadow on) — 요청당 파일 I/O의 처리량 델타	12.3

핵심 코드

오케스트레이터가 부하 중 서버 프로세스 트리의 CPU%·RSS를 샘플링하는 부분 — psutil 객체를 pid별로 **지속 보관**해야 CPU 델타가 잡힌다(매번 새 객체를 만들면 CPU 델타가 항상 0으로 측정된다).

```
for pid, p in tracked.items(): # 프라이밍된 Process 객체를 재사용
    cpu_total += p.cpu_percent(None) # 직전 호출 이후의 CPU% (누적 아님)
    rss_total += p.memory_info().rss
```


전체 코드는 docs_practice/chapter12/run_chapter12.py, 부하 시나리오는 docs_practice/chapter12/code/12-1-load-test.py 참고

실행 중 확인 체크리스트

- ☐ 시나리오 0에서 app.py sha256 일치가 출력되는가 — 10장과 바이트 동일이라는 증명
- ☐ bootstrap 지문이 96f6b6b3... 이고 champion 로드 검증 예측이 6.0인가
- ☐ 모든 시나리오에서 failures=0 인가 — 부하 하에서도 전 요청이 옳은 답(6.0)
- ☐ 워커 1개 세 시나리오(baseline-ramp-spike)의 RPS가 ≈ 2700 에서 천장을 치는가(CPU 병목)
- ☐ 워커 2개(scale)에서 RPS가 오르고 CPU가 한 코어→두 코어로 느는가
- ☐ shadow on(io)에서 RPS가 내려가는가 — 요청당 부수 작업의 값
- ☐ 마지막 줄이 CH12_RUN_PASS 인가

산출물 안내

산출물	내용	용도
ch12_load_report.json	시나리오별 실측(지연·RPS·CPU·RSS)·재조립 sha·지문·관찰	이 장 인용 수치의 증거 파일
ch12_bootstrap_summary.json	재구성 판정(승격·반려·별칭)·지문·champion 검증 예측	9·10장 판정과의 대조
mlflow.db · mlruns/ · ch12_shadow_log.jsonl	로컬 레지스트리·모델 실물·shadow 로그	재실행이면 다시 생기므로 커밋 제외

실행 결과(실제)

실제 실행 콘솔 출력의 발췌다.

```
== 시나리오 0: 서빙 앱 재조립 확인 ==
  app.py sha256 일치 (10장 파일과 바이트 동일)
== 시나리오 1: 레지스트리 재구성(bootstrap) ==
[bootstrap] 훈련 쌍 6개, 지문 96f6b6b3ce9dcdd1...
[bootstrap] champion=v1, challenger=v2
[bootstrap] champion 로드 검증 예측=6.0 (champion=v1)
== 부하: baseline (workers=1, shadow=False, u=20, t=12s) ==
  requests=27496 failures=0 rps=2498.3 p50=7.0ms p95=11.0ms p99=18.0ms cpu_peak=106.3%
  rss_peak=312.3MB
== 부하: scale (workers=2, shadow=False, u=20, t=12s) ==
  requests=34713 failures=0 rps=3153.0 p50=6.0ms p95=9.0ms p99=12.0ms cpu_peak=181.3%
  rss_peak=610.1MB
CH12_RUN_PASS
```

지연·RPS·CPU 수치는 **특정 실행의 스냅샷**이다 — 재실행하면 값이 변한다. 변하지 않는 것은 구조적 결론이다: 단일 워커의 처리량 천장, 워커 증설 시 처리량 상승, shadow on 시 처리량 하락, 그리고 전 시나리오에서 `failures=0` (부하 하에서도 정확성 유지). 표 12.5가 다섯 시나리오의 전체 스냅샷이고, 아래는 단계 증가(ramp)에서 동시 사용자가 늘 때 p95가 따라 오르는 궤적이다(10초 슬라이딩 창 기준).

표 12.5b 단계 증가 시 동시성 대 p95 지연(ramp, 창별 스냅샷)

동시 사용자	10	20	30	40	50
p95 지연	5ms	9ms	13ms	16ms	18ms

사용자가 늘 때 p95가 대체로 우상향한다(창별 표본이라 다소 변동한다) — 처리량이 이미 천장이라, Little's law대로 동시성이 늘면 지연이 따라 늘기 때문이다. “무릎점”이 급격한 꺾임이 아니라 완만한 상승으로 나타난 것은 이 부하 구간(사용자 50까지)이 아직 붕괴 이전이라는 뜻이다 — 부하를 더 늘리면 큐가 누적되며 꺾임이 나타난다.

워크드 해석: SLO로 용량을 읽는다

이 수치들은 결국 하나의 결정으로 모인다 — “이 서버는 몇 명까지 감당하는가”. 그 답은 SLO를 정해야 나온다. 예를 들어 SLO를 “p99 < 50ms”로 잡으면(설명을 위한 예시 값), 워커 하나 기준으로 기준 부하(20명, p99 18ms)와 단계 증가(50명, p99 32ms)는 통과하지만 스파이크(100명, p99 66ms)는 초과한다. 즉 이 SLO에서 워커 하나의 실효 용량은 **사용자 50명과 100명 사이 어딘가**다. 더 정밀한 용량이 필요하면 그 구간을 좁혀(60·70·80명) 다시 재면 된다. 여기에 12.4의 결과를 겹치면 대응이 보인다 — 목표 용량이 100명이라면 워커를 늘려(스케일아웃) 스파이크의 p99를 SLO 안으로 되돌리는 것이 1순위 조치다(표 12.6). 부하 수치가 “빠르다/느리다”의 인상이 아니라 “SLO 대비 몇 명까지”의 용량 판정으로 바뀌는 지점이 이 해석이다.

병목 원인 추정 및 개선 우선순위표(정본 산출물)

실측에서 병목을 추정하고 개선의 우선순위를 매긴 것이다. 우선순위는 **실측 근거의 강도와 적용 비용**으로 정한다.

표 12.6 병목 원인 추정 및 개선 우선순위표

순위	개선	실측 근거	기대 효과	비용
1	워커 증설 (스케일아웃)	워커 1→2에서 RPS 1.26배·CPU 한 코어→두 코어	처리량 천장 상향	낮음(설정·메모리× 워커)
2	요청 경로에서 로깅 분리	shadow on에서 RPS 0.68배	처리량 회복	낮음(비동기 큐· 배치)
3	부하 생성기 분리 후 재측정	생성기가 서버와 코어 경합 (12.2·12.4)	한계값 정확도 ↑	중(별도 machine)
4	모델 경량화 (ONNX 등)	현재 상수 예측기라 CPU 병목은 프레임워크 오버헤드	무거운 모델일 때 유효	중(변환+등가 검증)

우선순위 4를 낮게 둔 이유는 다음과 같다 — 흔히 “모델을 가볍게”가 첫 대응으로 제시되지만, 이 실측에서 병목은 모델 연산이 아니라 단일 워커의 CPU였다. 측정이 우선순위를 뒤집은 것이다 (12.3의 “가정하지 말고 측정하라”).

관찰

처리량 천장이 곧 CPU 병목이었다. 사용자 20→50→100에 RPS는 2498.3·2648.5·2788.5로 제자리, 지연만 다섯 배가 되었다. 워커 하나가 한 코어(106.3%)를 채운 것이 원인이고, Little’s law가 이 지연 증가를 정확히 예측했다($W=L/\lambda$).

워커 증설이 천장을 올렸다. 워커 2개에서 RPS 3153.0(1.26배)·CPU 두 코어(지연은 이 스냅샷에선 p50 7→6ms로 소폭 개선, 낮은 동시성이라 폭은 작음). 크기는 부하·환경에 달렸지만 효과의 부호는 확정적이다.

요청당 부수 작업은 비용을 수반한다. shadow on에서 RPS가 0.68배로 감소했다 — 요청 경로에 side effect를 추가할 때의 비용을 추정이 아니라 실측으로 확인했다.

부하 하에서도 정확성은 유지됐다. 전 시나리오 `failures=0` — 성능이 저하되어도(지연이 늘어도) 답은 여전히 6.0이었다. 지연 증가와 오답은 다른 고장이고, 부하 테스트는 둘을 함께 확인해야 한다.

부하 실측을 회귀 게이트로(10장 CI 연결)

이 실습의 증거 JSON에는 재실행마다 변하는 값(지연·RPS)과 변하지 않아야 하는 값(전 시나리오 `failures==0`, 응답 6.0, 지문 일치)이 함께 있다. 뒤쪽 결정적 값은 10장의 CI 게이트에 그대로 엮을 수 있다 — push마다 짧은 부하를 걸어 “부하 하에서도 오류 0·응답 6.0”을 확인하면, 성능 회귀(예: 어떤 변경이 처리량을 절반으로 감소시킴)나 정확성 회귀(부하 중 오답)를 배포 전에 검출한다. 표 12.1의 세 번째 목적(회귀 방지)이 10.5의 “생략을 구조적으로 제거하는 CI”와 만나는 지점이다. 다만

비결정 값(지연 임계)을 게이트로 사용하려면 machine-부하가 고정돼야 하므로, 회귀 게이트의 임계는 절대값이 아니라 "직전 기준선 대비 배수"로 두는 것이 안전하다(11.2 모니터링 지표 설계표의 기준선 사고와 같다).

실습과 운영의 간극(축소 지점 목록)

단일 노트북에서 부하 생성기와 서버가 코어를 공유한다 — 운영은 생성기를 분리하고 서버를 별도 인프라에 둔다(절대 수치보다 상대 비교에 무게를 둔 이유).

champion이 상수 예측기라 병목이 프레임워크 오버헤드다 — 무거운 실모델이라면 병목이 모델 연산으로 옮겨 가고, 경량화(보충)의 우선순위가 오른다.

워커 증설은 한 machine 안의 수평 확장이다 — 여러 machine-오토스케일(HPA)-로드밸런서는 범위 밖(보충).

부하가 12~24초의 짧은 실측이다 — 운영의 지속 부하-장시간 안정성(메모리 누수의 느린 발현 등)은 더 긴 부하로 봐야 한다.

위험 식별(평가표 연계)

위험	이 실습에서의 증거	운영 대응
병목 오진	상수 모델인데 CPU 포화 (프레임워크)	가정 없이 CPU-메모리-I/O를 함께 측정
측정 도구 병목	생성기-서버 코어 경합	생성기 분리-분산 부하
요청 경로 side effect	shadow on RPS 0.68배	로깅-외부 호출을 비동기-배치로
급증 시 미검증	스파이크는 실측, 실환경 급증은 규모 축소	예상 급증 배수로 사전 부하-스케일링 설계
장시간 안정성 미검증	12~24초 부하 — 느린 누수는 미발현	내구(soak) 테스트로 장시간 RSS-지연 관찰

보충(전문가 트랙 포인터)

Kubernetes HPA와 custom metric: 워커 증설의 클러스터판이다 — CPU-메모리 기반 자동 확장이 기본이고, 지연-큐 길이 같은 서비스 고유 신호로 확장하려면 custom metric(예: 초당 요청-컨슈머 lag)을 노출해야 한다. 이 장의 "무엇이 병목인가"가 곧 "무엇을 스케일 기준으로 삼을 것인가"로 이어진다.

ONNX 변환과 모델 경량화 검증: 병목이 모델 연산(CPU)일 때의 대응이다. 변환으로 추론을 가볍게 하되, 변환 전후가 **같은 입력에 같은 출력**을 내는지 등가 검증이 필수다(10장 환경 동등성 실측의 경량화판 — 빨라졌는데 답이 달라졌다면 최적화가 아니라 회귀다).

캐싱 전략과 정확성 리스크: 동일 입력이 반복되면 응답 캐시가 지연을 없앤다. 그러나 입력 키 설계를 잘못하면 다른 입력에 같은 답을 주고, 모델이 갱신(재학습, 11장)돼도 캐시가 낡은 답을 계속 서빙한다 — 캐시는 성능 최적화이자 정확성 리스크이므로 무효화(invalidation) 규칙과 함께 설계한다.

핵심 용어

처리량(throughput/RPS): 초당 완료 요청 수 — 부하 하 용량의 척도

지연 분위수(p50/p95/p99): 평균만으로 확인하기 어려운 느린 요청 구간을 나타내는 지표 — p99는 요청의 99%가 이 값 이하였음을 뜻한다.

포화(saturation): 골든 시그널 넷째 — 자원(CPU·메모리)이 얼마나 찼는가

Little's law($L=\lambda W$): 동시성=처리량×지연 — 닫힌 루프에서 사용자 수= L , $W=L/\lambda$

knee(무릎점): 부하 증가 시 지연이 급격히 꺾이는 지점 — 한계의 신호

가상 사용자(HttpUser)·spawn-rate: Locust가 부하를 기술하는 단위

CPU/메모리/I/O 병목: 처리량 천장+CPU 포화 / RSS 상승 / CPU 여유+지연 정체 — 증상으로 구분

스케일업과 스케일아웃: 큰 machine 하나와 작은 인스턴스 여럿 — stateless라야 수평 확장 비용이 낮다

워커(-workers): 서빙을 여러 프로세스로 — 독립 GIL로 단일 코어 한계를 푼다

검수 질문 한 줄 요약

발주·검수 문서에서 이 장은 세 질문으로 압축된다: ① 예상 피크·급증 부하와 그 부하에서의 지연 목표(p95-p99)가 정의되고 **부하 테스트 보고서로 검증**되는가(요구 수치만 있고 실측이 없으면 미검증 주장), ② 병목 유형(CPU·메모리·I/O)이 측정으로 식별되고 개선 우선순위가 실측 근거를 갖는가(짐작으로 최적화하면 엉뚱한 곳에 투자), ③ 재난 급증 대응이 스케일링 실측(워커·인스턴스·파티션)으로 뒷받침되는가(14장 범위표 연계 — 실측 없는 “재난 대응”은 주장). 셋 다 없으면 그 시스템의 용량은 개통 첫날 트래픽으로 처음 시험된다.

다음 장 예고

부하와 스케일까지 다뤘지만, 지금까지 감시·배포·부하의 대상은 늘 “모델”이었다. 그런데 모델은 피쳐(feature) 위에 서 있다 — 8장에서 피쳐 스토어로 만든 그 값들이 여러 모델·여러 팀에 공유되기 시작하면, 새로운 관리 문제가 생긴다: 이 피쳐는 누가 정의했고 누가 쓰는가, 민감한 피쳐에 아무나 접근해도 되는가, 피쳐 하나를 바꾸면 어느 모델이 영향을 받는가. 13장은 이 **피쳐 거버넌스**를 다룬다 — 피쳐 카탈로그, 접근 제어와 사용 이력, 품질 점검과 변경 관리. 14장이 “피쳐가 다중·다소비자가 되면 카탈로그·접근 로그·변경 관리가 필요하다”며 예고한 그 주제다.

참고문헌

- Locust. (n.d.). *Locust Documentation*. <https://docs.locust.io/en/stable/> (readthedocs rate-limit로 429 응답이 있으나 문서 실재)
- Locust. (n.d.). *Locust — A modern load testing framework*. <https://locust.io/>
- Uvicorn. (n.d.). *Deployment*. <https://uvicorn.dev/deployment/>
- FastAPI. (n.d.). *Server Workers - Uvicorn with Workers*. <https://fastapi.tiangolo.com/deployment/server-workers/>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering* — Ch. 6: Monitoring Distributed Systems(golden signals-saturation). O'Reilly. <https://sre.google/sre-book/monitoring-distributed-systems/>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering* — Ch. 21: Handling Overload(the pitfalls of QPS-per-customer limits-load shedding). O'Reilly. <https://sre.google/sre-book/handling-overload/>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering* — Ch. 22: Addressing Cascading Failures(graceful degradation-load shedding-queue management). O'Reilly. <https://sre.google/sre-book/addressing-cascading-failures/>
- Grafana Labs. (n.d.). *Load test types* (k6 documentation). <https://grafana.com/docs/k6/latest/testing-guides/test-types/>
- Little, J. D. C. (1961). A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research*, 9(3), 383–387. <https://doi.org/10.1287/opre.9.3.383>
- Rodola, G. (n.d.). *psutil — process and system utilities*. <https://pypi.org/project/psutil/>